

AutoMapper Complex Mapping in C# with Examples

In this article, I am going to discuss **AutoMapper Complex Mapping in C#** with Examples. Please read our previous article before proceeding to this article where we discussed the basics of [Automapper in C#](#) with Examples. At the end of this article, you will understand what is AutoMapper Complex Mapping and when and how to use AutoMapper Complex Mapping in C# with Examples.

What is AutoMapper Complex Mapping in C#?

When both types (Source and Destination) involved in the mapping contain properties of the complex type then in such scenarios we need to use the AutoMapper Complex Mapping in C#. Let us understand AutoMapper Complex Mapping with an example. We are going to use the following four classes for this demo.

```
public class Employee
{
    public string Name { get; set; }
    public int Salary { get; set; }
    public string Department { get; set; }
    public Address Address { get; set; }
}

public class EmployeeDTO
{
    public string Name { get; set; }
    public int Salary { get; set; }
    public string Department { get; set; }
    public AddressDTO Address { get; set; }
}

public class Address
{
    public string City { get; set; }
    public string State { get; set; }
    public string Country { get; set; }
}

public class AddressDTO
{
    public string City { get; set; }
    public string State { get; set; }
    public string Country { get; set; }
}
```

Create a class file with the name Address.cs and then copy and paste the following code into it.

```
namespace AutoMapperDemo
{
    public class Address
    {
        public string City { get; set; }
        public string State { get; set; }
        public string Country { get; set; }
    }
}
```

Create a class file with the name Employee.cs and then copy and paste the following code into it.

```
namespace AutoMapperDemo
{
```

```

public class Employee
{
    public string Name { get; set; }
    public int Salary { get; set; }
    public string Department { get; set; }
    public Address Address { get; set; }
}

```

Create a class file with the name AddressDTO.cs and then copy and paste the following code into it.

```

namespace AutoMapperDemo
{
    public class AddressDTO
    {
        public string City { get; set; }
        public string State { get; set; }
        public string Country { get; set; }
    }
}

```

Create a class file with the name EmployeeDTO.cs and then copy and paste the following code into it.

```

namespace AutoMapperDemo
{
    public class EmployeeDTO
    {
        public string Name { get; set; }
        public int Salary { get; set; }
        public string Department { get; set; }
        public AddressDTO Address { get; set; }
    }
}

```

Example to Understand AutoMapper Complex Mapping in C#:

Our requirement is to map the **Employee** object with the **EmployeeDTO** object. To make this simple, here we created both classes with the same property names. But the thing that we need to keep in mind here is, we created the address property as a complex type. Next, create a class file with the name **MapperConfig.cs** and then copy and paste the following code into it. Here, we are creating one static method called **InitializeAutomapper** and inside this method, we are writing all the mapping code. As you can see, here, we are mapping Employee with EmployeeDTO class.

```

using AutoMapper;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {

```

```

        //Provide all the Mapping Configuration
        var config = new MapperConfiguration(cfg => {
            //Configuring Employee and EmployeeDTO
            cfg.CreateMap<Employee, EmployeeDTO>();
        });
        //Create an Instance of Mapper and return that Instance
        var mapper = new Mapper(config);
        return mapper;
    }
}

```

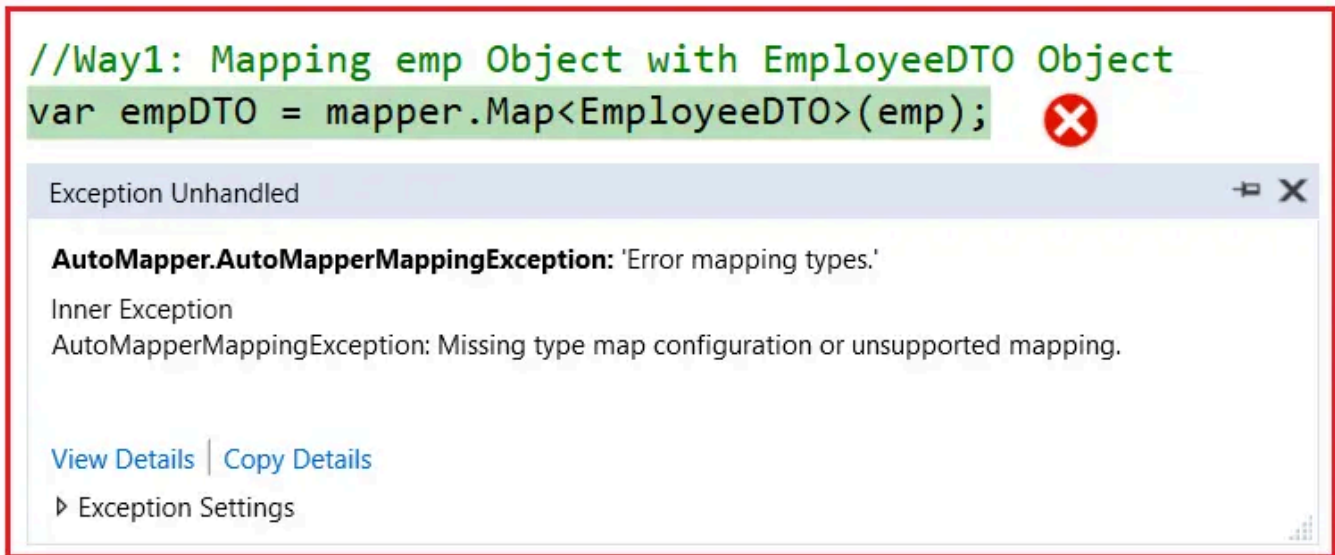
Next, modify the Main method of the Program class as follows. The following example code is self-explained, so please go through the comment lines for a better understanding of the code.

```

using System;
namespace AutoMapperDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating and Instance of Address Entity
            Address empAddress = new Address()
            {
                City = "Mumbai",
                State = "Maharashtra",
                Country = "India"
            };
            //Creating and Instance of Employee Entity
            Employee emp = new Employee
            {
                Name = "James",
                Salary = 20000,
                Department = "IT",
                Address = empAddress
            };
            //Initialize the AutoMapper
            var mapper = MapperConfig.InitializeAutomapper();
            //Way1: Mapping emp Object with EmployeeDTO Object
            var empDTO = mapper.Map<EmployeeDTO>(emp);
            //Way2: Mapping emp Object with EmployeeDTO Object
            //var empDTO = mapper.Map<Employee, EmployeeDTO>(emp);
            Console.WriteLine("Name:" + empDTO.Name + ", Salary:" + empDTO.Sal
            Department:" + empDTO.Department);
            Console.WriteLine("City:" + empDTO.Address.City + ", State:" +
            empDTO.Address.State + ", Country:" + empDTO.Address.Country);
            Console.ReadLine();
        }
    }
}

```

Now, when you run the application, you will get the following runtime exception.



Now, if you go to the inner exception and check the message property, then it clearly shows that the mapping type configuration is missing for Address and AddressDTO and the Exception message will be **“Missing type map configuration or unsupported mapping.\r\n\r\nMapping types:\r\nAddress -> AddressDTO\r\nAutoMapperDemo.Address -> AutoMapperDemo.AddressDTO”**

This is because, in our Mapper Configuration, we have specified the Mapper for Employee and EmployeeDTO but we have not specified the Mapping for Address and AddressDTO type.

How to Solve the Above Problem?

In order to solve the above problem, you need to configure the mapping between the Address and AddressDTO along with the Employee and EmployeeDTO Mapping. So, modify the **InitializeAutomapper** method of the **MapperConfig** class as follows.

```
using AutoMapper;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Provide all the Mapping Configuration
            var config = new MapperConfiguration(cfg => {
                //Configuring Address and AddressDTO
                cfg.CreateMap<Address, AddressDTO>();
                //Configuring Employee and EmployeeDTO
                cfg.CreateMap<Employee, EmployeeDTO>();
            });
            //Create an Instance of Mapper and return that Instance
            var mapper = new Mapper(config);
            return mapper;
        }
    }
}
```

```
}  
}
```

With the above changes in place, now, run the application, and then you should get the output as expected as shown in the below image.

```
Name:James, Salary:20000, Department:IT  
City:Mumbai, State:Maharashtra, Country:India
```

What Happens If We Change the Complex Type Property Name?

Let us understand this with an example. If you see, we have the complex property address within the EmployeeDTO class and the same in the case of the Employee class. Let's change the complex property name address to AddressDTO in the EmployeeDTO. So modify the EmployeeDTO class as follows.

```
namespace AutoMapperDemo  
{  
    public class EmployeeDTO  
    {  
        public string Name { get; set; }  
        public int Salary { get; set; }  
        public string Department { get; set; }  
        public AddressDTO AddressDTO { get; set; }  
    }  
}
```

Next, modify the Main method of the Program class as follows.

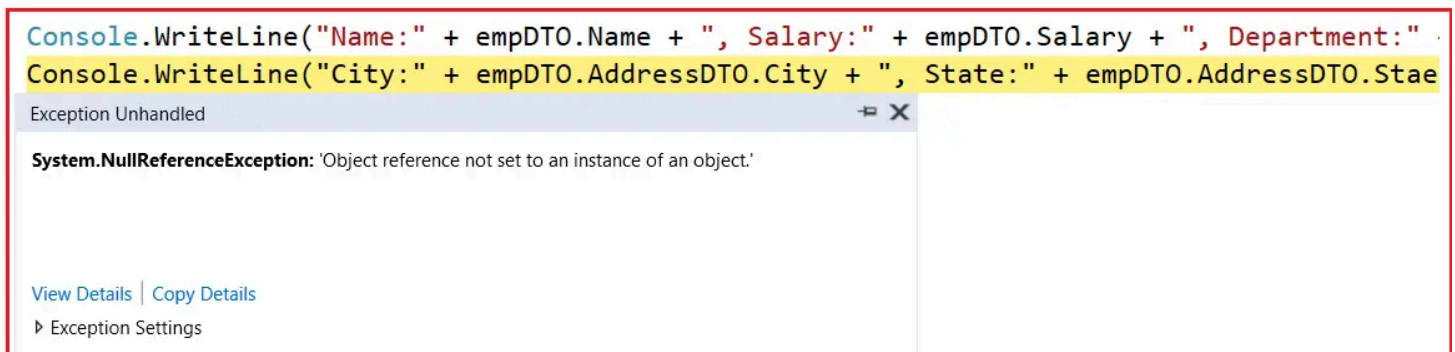
```
using System;  
namespace AutoMapperDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            //Creating and Instance of Address Entity  
            Address empAddress = new Address()  
            {  
                City = "Mumbai",  
                State = "Maharashtra",  
                Country = "India"  
            };  
            //Creating and Instance of Employee Entity  
            Employee emp = new Employee  
            {  
                Name = "James",  
                Salary = 20000,  
                Department = "IT",  
                Address = empAddress  
            }  
        }  
    }  
}
```

```

};
//Initialize the AutoMapper
var mapper = MapperConfig.InitializeAutomapper();
//Way1: Mapping emp Object with EmployeeDTO Object
var empDTO = mapper.Map<EmployeeDTO>(emp);
//Way2: Mapping emp Object with EmployeeDTO Object
//var empDTO = mapper.Map<Employee, EmployeeDTO>(emp);
Console.WriteLine("Name:" + empDTO.Name + ", Salary:" + empDTO.Sal
Department:" + empDTO.Department);
Console.WriteLine("City:" + empDTO.AddressDTO.City + ", State:" +
empDTO.AddressDTO.State + ", Country:" + empDTO.AddressDTO.Country);
Console.ReadLine();
    }
}
}

```

Now run the application. It should give you the following `NullReferenceException`. This is because the Complex Property `AddressDTO` is not initialized by `AutoMapper` from the `Employee` object address Property. As a result, the default value of the complex type is `NULL` and on a `NULL` object, when we access any property or any method, we will get `NULL Reference Exception` which is shown in the below image when we are trying to access the `City`, `State`, and `Country` property using the `AddressDTO` complex property whose value is `NULL`.



How to Solve the Above Problem?

To solve the above problem, we need to map the **Address** property of the `Employee` object to the **AddressDTO** property of the `EmployeeDTO` object in the mapper configuration using the `ForMember` method. To do so, modify the **InitializeAutomapper** method of the **MapperConfig** class as follows.

```

using AutoMapper;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Provide all the Mapping Configuration
            var config = new MapperConfiguration(cfg => {
                //Configuring Address and AddressDTO
                cfg.CreateMap<Address, AddressDTO>();
            });
        }
    }
}

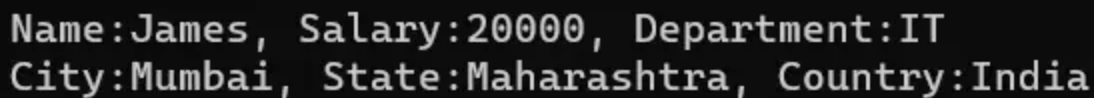
```

```

        //Configuring Employee and EmployeeDTO
        cfg.CreateMap<Employee, EmployeeDTO>()
        //Provide Mapping Information for AddressDTO and address
        .ForMember(dest => dest.AddressDTO, act => act.MapFrom(src =>
src.Address)); ;
    });
    //Create an Instance of Mapper and return that Instance
    var mapper = new Mapper(config);
    return mapper;
}
}
}

```

With the above changes in place, now run the application and it will give you the result as expected as shown in the below image.



```

Name:James, Salary:20000, Department:IT
City:Mumbai, State:Maharashtra, Country:India

```

What happens if the Complex Type Property Names are Different?

Let us understand this with an example. Let's modify the property names in the **AddressDTO** class as shown below.

```

namespace AutoMapperDemo
{
    public class AddressDTO
    {
        public string EmpCity { get; set; }
        public string EmpState { get; set; }
        public string Country { get; set; }
    }
}

```

Next, modify the Main method of the Program class as follows.

```

using System;
namespace AutoMapperDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating and Instance of Address Entity
            Address empAddress = new Address()
            {
                City = "Mumbai",
                State = "Maharashtra",
                Country = "India"
            }
        }
    }
}

```

```

};
//Creating and Instance of Employee Entity
Employee emp = new Employee
{
    Name = "James",
    Salary = 20000,
    Department = "IT",
    Address = empAddress
};
//Initialize the AutoMapper
var mapper = MapperConfig.InitializeAutomapper();
//Way1: Mapping emp Object with EmployeeDTO Object
var empDTO = mapper.Map<EmployeeDTO>(emp);
//Way2: Mapping emp Object with EmployeeDTO Object
//var empDTO = mapper.Map<Employee, EmployeeDTO>(emp);

    Console.WriteLine("Name:" + empDTO.Name + ", Salary:" + empDTO.Sal
Department:" + empDTO.Department);
    Console.WriteLine("City:" + empDTO.AddressDTO.EmpCity + ", State:'
empDTO.AddressDTO.EmpState + ", Country:" + empDTO.AddressDTO.Country);
    Console.ReadLine();
}
}
}

```

When you run the application, it will not give us any error, but it will not map the **City** and **State** property as shown in the below output.

```

Name:James, Salary:20000, Department:IT
City:, State:, Country:India

```

This is because we mapped the **Address** object with the **AddressDTO** object, but we had not mapped the **City** and **State** properties with **EmpCity** and **EmpState** properties. Let's map the above two properties and see what happens. To map the above two properties we need to change the **InitializeAutomapper** method of the MapperConfig class as follows.

```

using AutoMapper;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Provide all the Mapping Configuration
            var config = new MapperConfiguration(cfg => {
                //Configuring Address and AddressDTO
                cfg.CreateMap<Address, AddressDTO>()
                    .ForMember(dest => dest.EmpCity, act => act.MapFrom(src => src
                    .ForMember(dest => dest.EmpState, act => act.MapFrom(src => sr

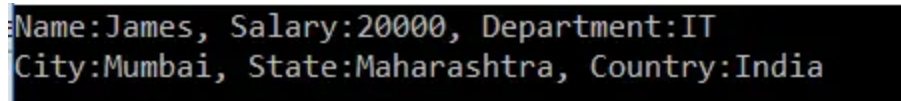
```

```

        //Configuring Employee and EmployeeDTO
        cfg.CreateMap<Employee, EmployeeDTO>()
        //Provide Mapping Information for AddressDTO and address
        .ForMember(dest => dest.AddressDTO, act => act.MapFrom(src =>
src.Address));
    });
    //Create an Instance of Mapper and return that Instance
    var mapper = new Mapper(config);
    return mapper;
}
}
}

```

With the above changes in place, now run the application and you will get the output as expected as shown in the below image.



```

Name:James, Salary:20000, Department:IT
City:Mumbai, State:Maharashtra, Country:India

```

In the next article, we will discuss how to map [Complex Types to Primitive Types in AutoMapper](#) in C# with Examples. In this article, I try to explain the **AutoMapper Complex Mapping in C#** with Examples. I hope this **AutoMapper Complex Mapping in C#** article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.